Evolutionary Computing 2026

Standard Assignment – Task II

Group 23

Anish Behera – 2918246
Siddhant Singh Bisht – 2923456
Munishwar Pradhan – 2922549
Eduardo Martin Quintana – 2705065

12th October 2026

Effects of Gaussian Mutation Step Size on Gecko Neuroevolution

1 INTRODUCTION

In this assignment, we evolve the weights of a neural-network controller to guide a fixed robot body from its initial position towards a target position [5]. The robot body is the gecko from the John Set, and the simulation environment is SimpleFlatWorld. The genotype consists of 138 real-valued weights, and the controller determines the target angles of the robot's six hinge joints. Fitness is measured as the Euclidean distance between the core module's final position and a target located 2 m away in the horizontal plane. The objective is to minimise this distance; a robot that remains stationary at its initial position has a fitness of 2 m.

Our evolutionary algorithm (EA) does not use crossover; Gaussian mutation is therefore the only source of variation during reproduction. Each offspring is generated by perturbing all 138 weights, with the magnitude of the perturbations controlled by the mutation step size σ. We investigate how this parameter affects the quality and consistency of the evolved controllers.

RQ1. How does the Gaussian mutation step size σ affect the final distance between the evolved gecko and the target? We compare σ = 0.1 and σ = 0.5, keeping all other experimental parameters constant.

RQ2. How does the Gaussian mutation step size affect the consistency of outcomes across independent runs?

H1. We hypothesise that the smaller mutation step size (σ = 0.1) will produce more gradual improvements and achieve a lower final distance than the larger step size (σ = 0.5), because smaller perturbations are expected to preserve more of the existing controller's behaviour.

Both EA configurations are also compared with random search under the same evaluation budget. This baseline allows us to assess whether evolutionary selection and mutation improve performance beyond independently sampling controllers from the initialisation distribution.

After 10,020 evaluations, both EA configurations outperform random search in terms of mean final distance. The mean distances are 0.565 m for σ = 0.1, 0.694 m for σ = 0.5, and 1.292 m for random search. Although the smaller mutation step size achieves a lower mean distance and makes faster early progress, its outcomes are more variable across the ten runs. The difference in final performance between the two EA configurations is not statistically significant (p = 0.162).

2 METHODS

2.1 Task

The robot is spawned at (0.0, 0.0, 0.1), and the target is positioned at (2.0, 0.0, 0.1). Each evaluation simulates 15 seconds of physics using MuJoCo [6] through the ARIEL framework [3]. Fitness is measured as the Euclidean distance between the robot's final core position and the target in the horizontal (x, y) plane. The objective is to minimise this distance. A controller that leaves the robot at its initial position obtains a fitness of 2 m, whereas a controller that reaches the target obtains a fitness of 0 m.

2.2 Controller

The controller is a fully connected feedforward neural network with 17 input units, 6 hidden units, and 6 output units. The hyperbolic tangent (tanh) activation function is applied to both the hidden and output layers. The output values are scaled to the interval $[-\pi/2, \pi/2]$ to represent the target angles of the robot's six hinge joints. The genotype consists of 138 real-valued weights across the two weight matrices, calculated as 17 × 6 + 6 × 6 = 138. No bias terms are used.

The network outputs are written directly to MuJoCo's data.ctrl array, meaning that each forward pass specifies the hinge control values for the current simulation step rather than accumulating changes to previous commands.

The input vector contains 13 values from data.qpos: three core-position coordinates, four components of the core-orientation quaternion, and six hinge angles. Four additional inputs provide information about the gait cycle and target direction. Two are periodic beat signals, $\sin(2\pi ft)$ and $\cos(2\pi ft)$, with frequency f = 1 Hz. The remaining two inputs represent the horizontal direction (dx, dy) from the robot's core to the target. The beat signals provide periodic information that can support gait generation, while the direction inputs allow the controller to account for the target's location. Preliminary single-seed experiments indicated that including the beat inputs improved progress towards the target.

2.3 Evolutionary algorithm

The evolutionary algorithm (EA) is implemented using the ARIEL evolutionary-computing framework, including its EA, EAOperation, Individual, Population, and FloatMutator components. The population size is μ = 20, and each generation produces λ = 20 offspring. Initially, each weight is sampled independently from a Gaussian distribution $N(0, 0.5^2)$.

Parent selection uses truncation selection: the best 50% of the population are eligible for reproduction. Each offspring begins as a copy of a randomly selected parent, sampled with replacement from this group. Consequently, a parent may produce multiple offspring or none. Gaussian mutation is then applied independently to every weight. For each weight w_i, the mutated value is $w_i' = w_i + \varepsilon_i$, where ε_i is sampled from $N(0, \sigma^2)$. The mutation probability is 1.0, meaning that every weight is perturbed. The mutation step size σ is set to either 0.1 or 0.5, depending on the experimental condition.

No crossover operator is used. Survivor selection follows an elitist (μ + λ) strategy [2], in which the 20 best individuals are selected from the combined population of parents and offspring. This ensures that the best fitness achieved so far is retained across generations.

Each run consists of 500 generations, with 20 individuals in the initial population and 20 offspring evaluated per generation. The total evaluation budget is therefore 20 + (500 × 20) = 10,020 evaluations per run. We selected 500 generations after a preliminary run of 300 generations on seed 1 showed that the σ = 0.5 configuration was still improving. However, as discussed in Section 3.6, improvement was not completely plateaued by the end of the final runs.

2.4 Baseline

Random search serves as the baseline for evaluating whether evolutionary selection and mutation improve controller performance. Each candidate controller is generated by independently sampling its 138 weights from the same initialisation distribution, $N(0, 0.5^2)$, used for the EA's initial population. Random search uses the same total evaluation budget of 10,020 evaluations per run. For comparison with the evolutionary runs, the best fitness found so far is recorded after each block of 20 evaluations. This allows the progress of random search to be compared with the EA configurations over an equivalent evaluation budget.

2.5 Experimental setup

Table 1 summarises the parameter settings used in the experiments. We evaluated three conditions: an EA with $\sigma = 0.1$, an EA with $\sigma = 0.5$, and random search. Each condition was evaluated across ten independent runs using seeds 1–10, resulting in 30 runs in total. The same seed was used across all three conditions to provide a consistent basis for comparison. The initial best fitness was identical across conditions for each seed, as shown in Table 4.

We compare the final best fitness values across the ten runs for each condition using a two-sided Mann–Whitney U test [4], with a significance level of $\alpha = 0.05$. Cliff's delta [1] is used to quantify the effect size. The p-values and effect sizes are obtained from the analysis script. Since each condition contains ten runs, we do not assume that the final fitness values follow a normal distribution. Statistical significance is interpreted alongside the effect sizes and descriptive statistics.

Each run is reproducible using its specified random seed. The seed is provided as a command-line argument and is used to initialise NumPy's random-number generator and the ARIEL framework's random-number generator through set_seed. Both initialisation steps are necessary to ensure that the random processes used by the experiment are controlled. A single run can be executed using the following command:

uv run A2_template_2026.py <seed> <sigma|random>

For example, uv run A2_template_2026.py 3 0.1 runs the EA with $\sigma = 0.1$ using seed 3, while uv run A2_template_2026.py 3 random runs random search using the same seed. The analysis script, plot_results.py, processes the experimental CSV files to generate the figures and statistical summaries reported in this paper.

Table 1: Experimental parameter settings. All parameters are identical between the two EA configurations except for the mutation step size σ.

Component	Setting
Framework and physics engine	ariel, MuJoCo, Python 3.12
Body	gecko, 6 hinges — constant
World	SimpleFlatWorld — constant
Spawn / target	(0, 0, 0.1) / (2, 0, 0.1)
Time per evaluation	15 s
Fitness	Euclidean (x, y) distance, minimised
Controller	$17 \to 6 \to 6$, tanh, no bias
Genotype	138 real values
Initialisation	each weight $\sim N(0, 0.5^2)$
μ / λ	20 / 20
Parent selection	truncation, best 50 %
Crossover	none
Mutation	Gaussian, p = 1.0, $\sigma \in \{0.1, 0.5\}$
Survivor selection	$(\mu + \lambda)$, elitist
Generations / budget	500 / 10,020 evaluations
Runs per setting	10 (seeds 1–10)
Baseline	random search, same budget

3 RESULTS AND DISCUSSION

3.1 EA settings outperform random search

After 10,020 evaluations per run, both evolutionary algorithm (EA) configurations achieved lower final distances than random search (Table 2). The EA with $\sigma = 0.1$ achieved a mean final distance of 0.565 m, compared with 0.694 m for $\sigma = 0.5$ and 1.292 m for random search. These results suggest that evolutionary selection and Gaussian mutation improve controller performance compared with independently sampling controllers from the initialisation distribution.

Table 2: Final best fitness after 500 generations, measured as distance from the target in metres. Lower values indicate better performance. Each setting includes ten independent runs.

Setting	Mean	Sd	Median	Best	Worst
EA, $\sigma = 0.1$	0.565	0.395	0.445	0.253	1.533
EA, $\sigma = 0.5$	0.694	0.214	0.792	0.300	0.899
Random search	1.292	0.090	1.318	1.156	1.388

Random search also improved its mean best fitness from 1.797 m initially to 1.292 m after the same evaluation budget. However, unlike the EA, random search does not use selection to favour better-performing controllers. The lower final distances achieved by both EA configurations therefore suggest that selection combined with mutation is beneficial for evolving the neural-network controller.

The statistical comparisons in Table 3 indicate substantial differences between both EA configurations and random search, based on the reported effect sizes and p-values. However, the difference between the two EA configurations was not statistically significant at the 0.05 significance level ($p = 0.162$). Thus, although $\sigma = 0.1$ achieved a lower mean final distance than $\sigma = 0.5$, the results do not provide sufficient statistical evidence to conclude that the smaller mutation step size is superior.

Table 3: Pairwise statistical comparisons of final best fitness across settings, with ten runs per setting.

Comparison	p	Cliff's δ
$\sigma = 0.1$ vs random search	0.003	+0.80
$\sigma = 0.5$ vs random search	< 0.001	+1.00
$\sigma = 0.1$ vs $\sigma = 0.5$	0.162	+0.38

The individual results in Table 4 provide further insight into the consistency of the two EA configurations. The $\sigma = 0.1$ configuration achieved a lower final distance than $\sigma = 0.5$ on seven of the ten seeds. However, its standard deviation was higher (0.395 m compared with 0.214 m), indicating greater variability in performance across runs. Overall, both EA configurations outperformed random search in terms of mean final distance, while the comparison between the two mutation step sizes remains inconclusive.

Table 4: Final best fitness for each seed, measured as distance from the target in metres. The initial best fitness is the best individual in the shared initial population for each seed.

Seed	Initial	σ = 0.1	σ = 0.5	Random
1	1.919	1.533	0.803	1.294
2	1.731	0.553	0.899	1.379
3	1.769	0.280	0.496	1.156
4	1.801	0.383	0.825	1.308
5	1.639	0.472	0.781	1.172
6	1.693	0.320	0.840	1.179
7	1.766	0.441	0.830	1.362
8	1.904	0.964	0.300	1.357
9	1.910	0.449	0.393	1.388
10	1.836	0.253	0.778	1.329
Mean	**1.797**	**0.565**	**0.694**	**1.292**

3.2 σ = 0.1 reaches a lower distance, but not significantly

The EA with σ = 0.1 achieved a lower mean final distance (0.565 m compared with 0.694 m) and a lower median final distance (0.445 m compared with 0.792 m) than the EA with σ = 0.5. It also achieved a lower final distance on seven of the ten seeds. However, the difference between the two EA configurations was not statistically significant (p = 0.162).

Consequently, the results do not provide sufficient statistical evidence to support H1, which predicted that σ = 0.1 would achieve a lower final distance than σ = 0.5. Although the observed results favour the smaller mutation step size, this difference should be interpreted as a trend rather than a statistically established advantage.

The difference between the two mutation settings was also smaller than the differences between either EA configuration or random search. The mean final distance differed by 0.129 m between the two EA configurations, whereas the differences between random search and the EA configurations were 0.727 m for σ = 0.1 and 0.598 m for σ = 0.5. This suggests that, in these experiments, using evolutionary selection and mutation had a greater effect on mean final performance than changing the mutation step size alone.

3.3 σ = 0.1 makes faster early progress

The difference in performance between the two EA configurations was most pronounced during the early stages of evolution. At generation 100, the mean best fitness was 0.884 m for σ = 0.1 and 1.195 m for σ = 0.5, a difference of 0.311 m (Table 5). By generation 500, this difference had decreased to 0.129 m. These results indicate that the smaller step size achieved faster early progress, although the performance gap narrowed as evolution continued.

Over the final 100 generations, the mean best fitness improved by 0.063 m for σ = 0.1 and 0.090 m for σ = 0.5. Thus, the σ = 0.5 configuration continued to improve at the end of the experiment, while σ = 0.1 also showed further improvement. Neither configuration can be assumed to have reached a complete plateau based on these changes alone.

The assignment brief identifies a plateau in fitness as the preferred stopping criterion. Since the σ = 0.5 configuration was still improving at generation 500, the fixed evaluation budget may have terminated the experiment before convergence was fully established. We discuss this limitation in Section 3.6.

Table 5: Mean best fitness across ten seeds at selected generations, measured as distance from the target in metres. Generation 0 represents the initial population. Lower values indicate better performance.

Generation	σ = 0.1	σ = 0.5	Random
0	1.797	1.797	1.797
100	0.884	1.195	1.424
250	0.723	0.921	1.376
400	0.627	0.785	1.327
500	0.565	0.694	1.292

3.4 σ = 0.1 shows the highest variability

Across the ten runs, the standard deviation of final distance was 0.395 m for σ = 0.1 and 0.214 m for σ = 0.5. Thus, the σ = 0.1 configuration showed greater variability in final performance. The individual results in Table 4 illustrate this variation. Eight of the ten runs with σ = 0.1 achieved final distances between 0.253 m and 0.553 m, while seed 1 resulted in a distance of 1.533 m and seed 8 resulted in 0.964 m. In contrast, the σ = 0.5 configuration achieved final distances between 0.300 m and 0.899 m, indicating a narrower spread across runs.

These results suggest that the mutation step size influences the balance between local refinement and exploration. A smaller step size, such as σ = 0.1, makes smaller changes to the neural-network weights, which may allow more gradual refinement of promising controllers. However, its performance may depend more strongly on the region of the search space reached during evolution. A larger step size, such as σ = 0.5, introduces larger changes to offspring and may allow the search to explore different regions of the solution space, although it can also make consistent refinement more difficult. This interpretation is consistent with the different convergence patterns shown in Figure 1, but the experiments do not directly establish the search mechanism responsible for the observed differences.

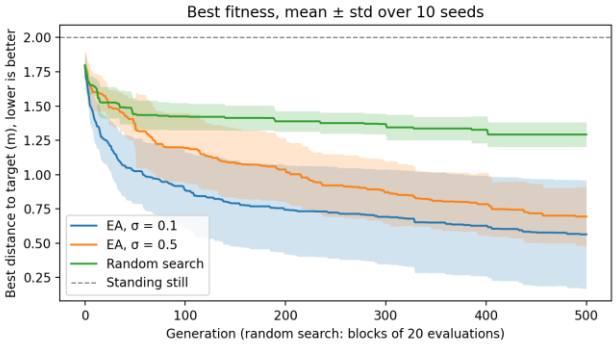


Figure 1: Best fitness per generation, shown as the mean ± standard deviation across ten seeds for each setting. For random search, each horizontal-axis step represents a block of 20 evaluations, allowing all three configurations to be compared over the same total budget of 10,020 evaluations. The dashed horizontal line at 2.0 m represents a robot that remains at its initial position. Lower fitness values indicate better performance.

Seed 1 illustrates the variation between the two configurations (Figure 2). Starting from the same initial population, σ = 0.5 achieved a final distance of 0.803 m, whereas σ = 0.1 achieved 1.533 m. This suggests that the smaller mutation step size does not consistently produce better outcomes, despite achieving the lower mean final distance overall.

The elitist (μ + λ) survivor-selection strategy retains the best individuals from the combined parent and offspring populations. Although this prevents the best fitness from deteriorating between generations, it does not guarantee that the population will continue to discover better solutions. If the search makes limited progress in a particular region, mutation must generate useful alternatives for further improvement. The results therefore suggest that σ = 0.1 is effective in many runs but can produce less favourable outcomes in some cases, whereas σ = 0.5 shows more consistent final distances across the ten runs. However, ten runs are insufficient to establish that σ = 0.5 would always be the safer choice when only one run is possible.

3.5 Population convergence

At generation 500, the average difference between the population mean fitness and the population best fitness was 0.094 m for σ = 0.1 and 0.104 m for σ = 0.5 (Figure 3). These values indicate that the population mean was relatively close to the best-performing individual in both configurations. The difference between the two settings was only 0.010 m, suggesting that their final population-level fitness gaps were similar.

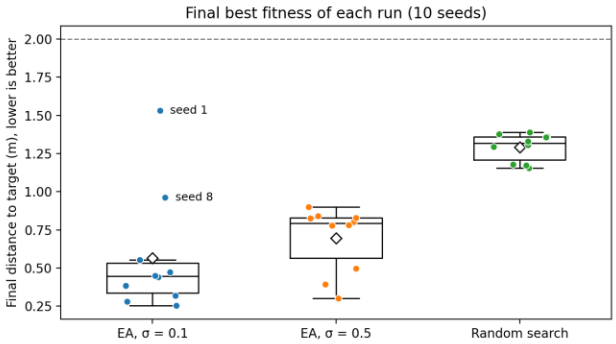


Figure 2: Final best fitness of every run (dots), with boxplots showing the median (line) and mean (diamond). Lower values indicate better performance. The two weakest σ = 0.1 runs (seeds 1 and 8) are labelled; the dashed line at 2.0 m represents a robot that remains at its initial position.

The results are broadly consistent with the possibility that the smaller mutation step size promotes more concentrated search around promising solutions. However, the difference between the two configurations is small, and the variation across independent runs should also be considered. Therefore, these results alone do not establish that σ = 0.1 produces stronger convergence or greater exploitation than σ = 0.5.

Figure 3 illustrates the evolution of population mean fitness across generations. The population mean decreases for both EA configurations, with σ = 0.1 generally achieving lower values throughout the run. This is consistent with the results for best fitness, although population mean fitness provides a complementary view of the overall population rather than only its best individual.

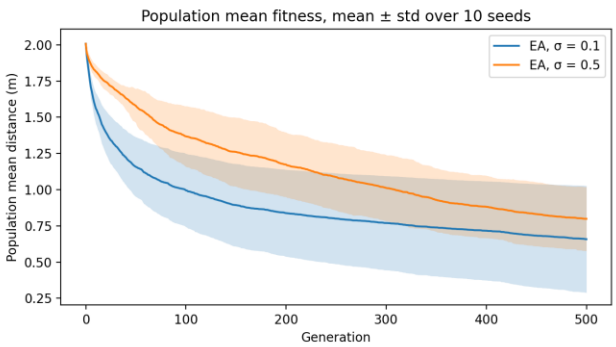


Figure 3: Population mean fitness per generation for the two EA configurations, shown as the mean ± standard deviation across ten seeds. Random search is omitted because its population mean is not directly comparable to the EA population mean.

3.6 Limitations

The first limitation is that the comparison was conducted using a fixed evaluation budget rather than a convergence-based stopping criterion. The σ = 0.5 configuration improved its mean best fitness by 0.090 m over the final 100 generations, indicating that it was still making progress when the experiment ended. Therefore, the results may not reflect the final performance that could be achieved with additional evaluations.

Second, each configuration was evaluated across ten independent runs. Although the results indicate clear performance differences between the EA configurations and random search, the difference between the two EA settings was not statistically significant. The relatively high variability observed for σ = 0.1 may have contributed to this result. Additional runs would help determine whether the observed difference between mutation step sizes is consistent.

Third, the experimental setup was limited to one robot body, one simulation environment, and one target position, with a population size of 20 and no crossover operator. Consequently, the findings may not generalise to other robot morphologies, environments, target positions, or EA configurations. Future experiments could investigate additional population sizes, mutation step sizes, crossover operators, and selection strategies to determine whether these choices improve controller performance.

Finally, fitness was defined solely as the final Euclidean distance between the robot's core position and the target. This objective does not account for other aspects of controller performance, such as movement efficiency, stability, the robot's final physical state, or potential damage during simulation. A controller that reaches the target may therefore receive a favourable fitness score despite exhibiting inefficient or undesirable behaviour. Future work could incorporate additional fitness components to evaluate not only target-reaching performance but also movement quality, stability, and physical integrity.

4 CONCLUSIONS

This study investigated how the Gaussian mutation step size affects the final distance achieved by an evolved gecko robot. Under an equal evaluation budget of 10,020 evaluations per run, both EA configurations outperformed random search in terms of mean final distance. The EA with σ = 0.1 achieved a mean distance of 0.565 m, while σ = 0.5 achieved 0.694 m, compared with 1.292 m for random search.

The difference between the two EA configurations was not statistically significant (p = 0.162). Nevertheless, σ = 0.1 achieved a lower mean and median final distance and outperformed σ = 0.5 on seven of the ten seeds. It also made faster progress during the early generations. However, its results were more variable, with a standard deviation of 0.395 m compared with 0.214 m for σ = 0.5. One run with σ = 0.1 also finished with a greater distance from the target than every random-search run.

Regarding the first research question, the results suggest that the smaller mutation step size achieved better average performance and faster early progress in these experiments, although its advantage over the larger step size was not statistically established. Regarding the second research question, σ = 0.5 produced more consistent final distances across the ten runs, whereas σ = 0.1 showed greater variability. Therefore, the choice of mutation step size involves a trade-off between average performance and consistency.

Future work could investigate a broader range of mutation step sizes, alternative population sizes, different initialisation strategies, and crossover operators. Longer experiments using a defined convergence criterion could also establish whether additional evaluations improve performance, particularly for σ = 0.5. Finally, evaluating movement efficiency, stability, and other aspects of robot behaviour could provide a more comprehensive assessment of controller quality.

5 USE OF AI

AI was used in accordance with the course guidelines on editorial support, rephrasing, and document structure. It assisted with improving the structure and grammar of the report, rephrasing text, and developing and checking the experiment-runner and statistical-analysis scripts. The team reviewed the code and executed the experiments. All reported experimental results were obtained from the team's runs.

REFERENCES

[1] N. Cliff. 1993. Dominance statistics: Ordinal analyses to answer ordinal questions. *Psychological Bulletin* 114, 3 (1993), 494–509.

[2] A. E. Eiben and J. E. Smith. 2015. *Introduction to Evolutionary Computing*. 2nd ed. Springer, Berlin, Heidelberg.

[3] Evolutionary Computing 2026. 2026. Course fork of the ARIEL framework. GitHub repository. https://github.com/AndrzejSzczepura/EvolutionaryComputing2026.

[4] H. B. Mann and D. R. Whitney. 1947. On a test of whether one of two random variables is stochastically larger than the other. *The Annals of Mathematical Statistics* 18, 1 (1947), 50–60.

[5] K. O. Stanley, J. Clune, J. Lehman, and R. Miikkulainen. 2019. Designing neural networks through neuroevolution. *Nature Machine Intelligence* 1, 1 (2019), 24–35.

[6] E. Todorov, T. Erez, and Y. Tassa. 2012. MuJoCo: A physics engine for model-based control. In *Proceedings of the 2012 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 5026–5033.